Module 9-1

Functional Verification Overview

cadence

This module spans 4 hours of lecture time.

Module Objectives

In this module, you will:

- Define functional verification.
- Debug using the waveform viewer, breakpoints and Xcelium simulator tool.

cadence

In this module, you will:

- Define functional verification.
- Write a simple SV testbench using the FSM example of a drink machine.
- Debug using the waveform viewer, breakpoints and Xcelium simulator tool.
- Recognize the three corner stones of modern verification flow: randomization, coverage, and assertions and why we need them.
- Define constraint-based randomization.
- Discuss coverage and other metrics (code coverage, functional coverage, SV cover groups).
- Define assertion-based verification (assertions and covers).
- Identify class-based testbenches, UVM and methodology.

You Also:

- Discuss verification planning, the concept of executable and dynamic verification plan and management, and mapping with metrics and coverage to the vPlan.
- Recognize the MDV Methodology and its different phases.
- Recognize formal verification and its use models
  - Exhaustive proof, Automated Formal Checks, Design Exploration, Equivalence Checking and many others.
- Identify Accelerated Verification (Emulation, Prototyping and multicore simulation).
- Identify the verification completeness problem – How do we know we are done?
- Identify the verification management aspect that is, “Do we know why it isn’t tested?” (bug rate, coverage, risk management)
- Identify the verification challenges and the process decisions made before tapeout.

What Is Functional Verification?

- Verification is the process, a continuous one at that, which runs throughout the design flow alongside each major phase.
- It ensures the functionality, timing, and integrity of the changing design throughout the process.
- It ensures a design's implementation meets the design specification requirements.
- Goal of verification:
  - Demonstrate the functional correctness of a design.
  - Attempt to find design errors.
  - Attempt to show that the design implements the specification.
- Which design characteristics are being verified:
  - Functionality
    - Does the design model perform all the operations it is required to?
    - Does the design model perform these operations correctly?
  - Timing
    - Does the design model meet the timing (performance) requirements required by the design specification?

3

cadence

Functional verification is the process that makes sure the coded design is functioning the right way according to the specification it followed for coding.

Verification is the process, a continuous one at that, which runs throughout the design flow alongside each major phase.

It ensures the functionality, timing, and integrity of the changing design throughout the process.

It ensures a design's implementation meets the design specification requirements.

The goal of verification is to:

- Demonstrate the functional correctness of a design.
- Attempt to find design errors.
- Attempt to show that the design implements the specification.

Now, what are the characteristics that are being verified during the verification process:

It verifies the functionality, as in:

- Is the design model performs all the operations it is required to?
- Is the design model performing these operations correctly?

It also checks the timing, that is, to see if the design model meets the timing as well as performance requirements required by the design specification.

Verification Methods

There are three methods to perform verification:

1. Simulation (dynamic)
   - Predominant verification method.
   - Only option for behavioral-level verification.
   - Involves developing testbenches and input vectors.
   - Harder to determine whether or not a verification suite completely checks all corner cases or not.
2. Formal (Static) – covered later
   - This derives from formal logic
   - Reasoning based on manipulation of formulas, hence static
   - The name formal can mean many things, e.g., any of the following:
     - Logic equivalency checking (LEC)
     - Model Checking – Functional formal verification (Formal Analysis)
     - Theorem proving
3. Emulation/Acceleration – covered later
   - Execute and debug RTL and software on custom hardware 1000's times faster than software simulator.
   - After compilation, the compile result is imported into the run-time interface to run and debug the design on the selected acceleration system.

cadence

This slide goes over the different methods of verification.

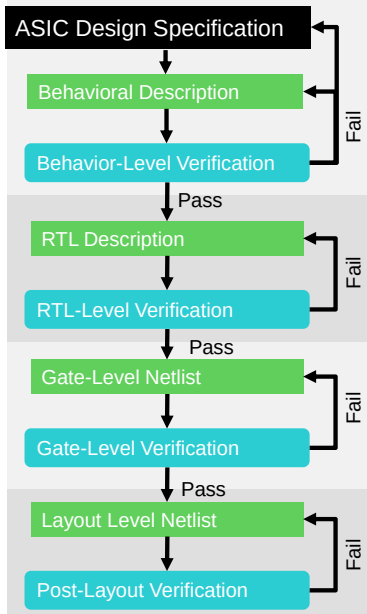
There are three methods followed:

1. The Simulation which is dynamic in nature
   - It is a Predominant verification method.
   - This is the Only option for behavioral-level verification.
   - It Involves developing testbenches and input vectors.
   - It is Harder to determine whether or not a verification suite completely checks all corner cases or not.

2. The Formal which is static in nature
   - This derives from `formal` logic.
   - It's Reasoning is based on manipulation of formulas, hence static.
   - The name `formal` can mean many things, e.g., any of the methods such as:
     - Logic equivalency checking (LEC)
     - Model Checking – Functional formal verification (Formal Analysis)
     - Or Theorem proving.

3. The third method is Emulation
   - In here the design is compiled for different modes, namely In-Circuit Emulation (ICE) and Simulation Acceleration (SA) according to the requirements.
   - After compilation, the compiled result is imported into the run-time interface to run and debug the design on the selected emulator system.
   - Then the RTL is executed and debugged along with software, on custom processor hardware 1000 times faster than software simulator.

Verification of Four Different Abstraction Levels



1. Behavioral-level verification
   - Does the behavioral description satisfy the requirements of the design specification?
2. RTL-level verification
   - Is the RTL level description functionally equivalent to the behavioral-level description?
3. Gate-level verification
   - Is the gate level netlist functionally equivalent to the behavioral-level description?
4. Post-gate-level verification
   - Is the gate-level netlist functionally equivalent to the gate-level description?

Language Description

Verification

5

cadence

We went over the different levels of abstraction on the design coding domain, and now we look at what we verify in each of these levels. This slide shows a flow diagram of an entire ASIC design specification process starting with behavioral phase till post gate level phase. And, in here we discuss what we verify in each phase.

So now, let's go over what we covered as to the coding aspects in each phase and then see what aspects we verify in each phase.

- At The behavioral phase:
  - You describe the system using mathematical equations.
  - You can omit timing – the system may simulate in zero time, like a software program.

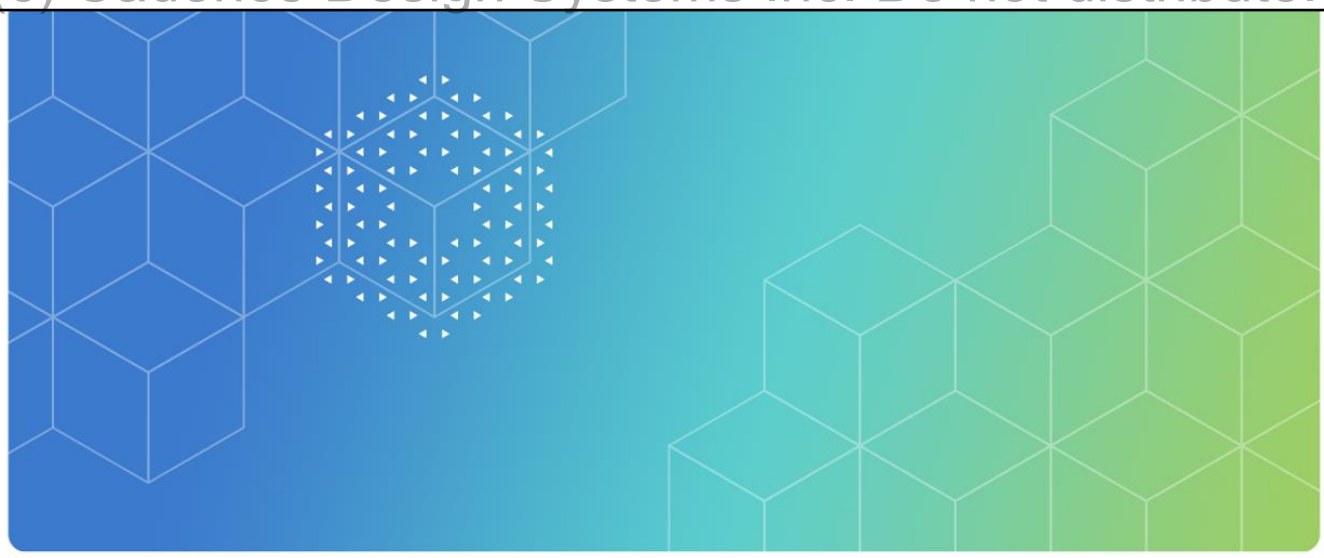
  At this phase, for the Behavioral level verification:
  - You check if the behavioral description satisfies the requirements of the design specification?

- At The Register Transfer Level (RTL):
  - You partition the system into combinational and sequential logic, using constructs and coding styles supported by logic synthesis.
  - You define timing in terms of cycles based on one or more defined clocks.

  At this phase, for the RTL level Verification:
  - You check if the gate level netlist functionally is equivalent to the behavioral-level description?
  - This is also considered as Gate level Verification

- At The structural level or gate level description:
  - You instantiate and interconnect predefined components.
  - You Can include vendor-provided macrocells.
  - You Can include logic primitives built into the language.

  At this phase, for the verification which is called as post gate level verification:
  - You check if the gate-level netlist is functionally equivalent to the gate-level description?



Module 9-2

Debug Techniques

cadence

This module talks about the interactive and post processing debug techniques.

Objectives

In this module, you will:

- List the different kinds of debug techniques.
- Debug an error using the SimVision™ GUI with relevant textual commands using the following steps:
  - Explore the drink machine SV design.
  - Execute the single-step *xrun* command to compile, elaborate and simulate the design.
  - Examine the error that displays.
  - View the error in the waveform as well as in the source window.
  - Analyze the method of correcting this error.
  - Fix the source code by opening the editor and re-compiling the code.
  - Re-simulate the design.

cadence

In this submodule, you will:

- List the different kinds of debug techniques.
- Debug an error using the SimVision™ GUI with relevant textual commands using the following steps:
- Explore the drink machine SV design.
- Execute the single-step *xrun* command to compile, elaborate and simulate the design.
- Examine the error that displays.
- View the error in the waveform as well as in the source window.
- Analyze the method of correcting this error.
- Fix the source code by opening the editor and re-compiling the code.
- Re-simulate the design.
- Examine the various textual commands used in the different stages listed above.

What Are Debug Techniques?

Debug is to rectify or resolve an error in the simulation and then accordingly rerun with the fix and complete the simulation process.

We have two kinds of debug techniques for a simulation tool.

1. **Interactive Debug**: Uses single stepping and break points.
2. **Post Processing Debug**: Done after you run the simulation.

cadence

So lets understand what is debug and the debug technique.

Debug is to rectify or resolve an error in the simulation and then accordingly rerun with the fix and complete the simulation process.

We have two kinds of debug techniques for a simulation tool.

1. **Interactive Debug** which Uses single stepping and break points.
2. **And Post Processing Debug** which is Done after you run the simulation.

Interactive Debug Process

This is the debug process performed while you are running your simulation through single stepping and breakpoints.

1. Setting breakpoints.

- Most common types of breakpoints:
  - Time-based – Stop the simulation on a specific line of code if we have reached a certain time.
  - Value-based – Stop the simulation on a specific line if ever a specific set of signals take on the desired value.

2. Single stepping allows users to run code in a controlled manner:
   - Access to all simulator data is a huge productivity boost for debug.
     - Can explore on the fly – Can examine the context of every line executing.
   - Can examine all variable values at the current simulation time.
   - Can selectively probe items of interest on the fly.
   - Step within the current thread, within all threads, into methods.

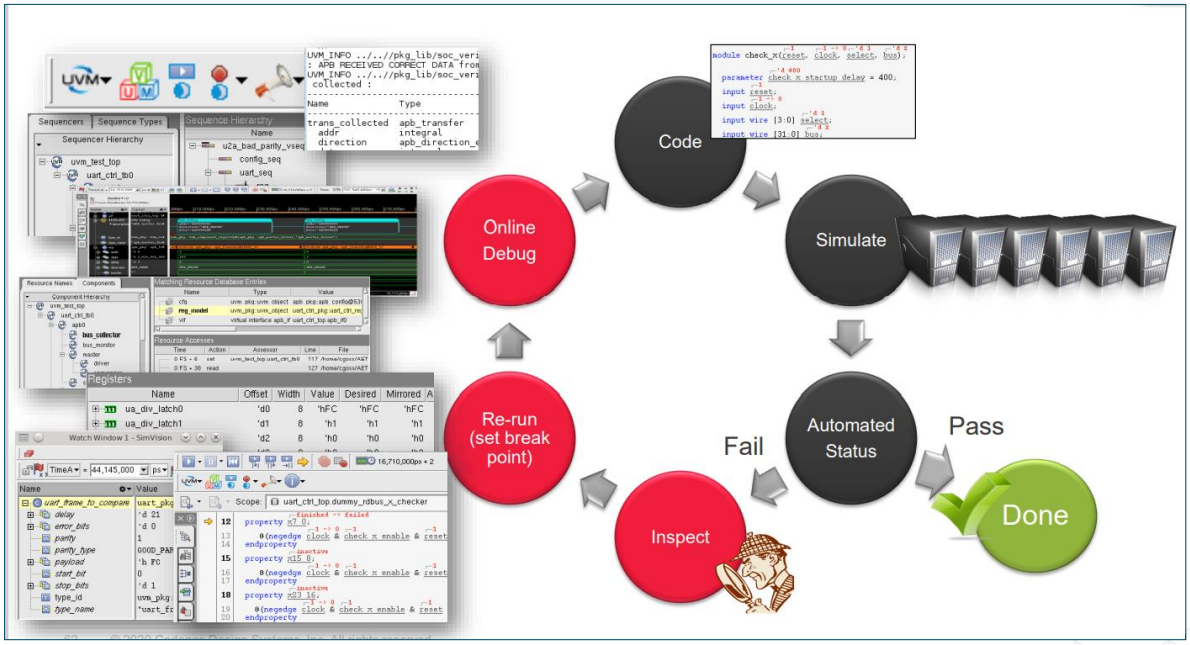
cadence

Let's understand the interactive debug process.

This is the debug process performed while you are running your simulation.

- Through single stepping and breakpoints.
- Let's talk about setting breakpoints.
- Most common types of breakpoints are:
  - Time-based – where you stop the simulation on a specific line of code if we have reached a certain time.
  - Value-based – where you stop the simulation on a specific line if ever a specific set of signals take on the desired value.
- Now, about the single-stepping process.
- Single stepping allows users to run code in a controlled manner:
  - Since it can access to all simulator data, it is a huge productivity boost for debugging.
    - Here, you can explore on the fly – you can examine the context of every line executing.
  - You can examine all variable values at the current simulation time.
  - You can selectively probe items of interest on the fly.
  - And step within the current thread, within all threads, into methods.

1. Interactive Debug Process (continued)

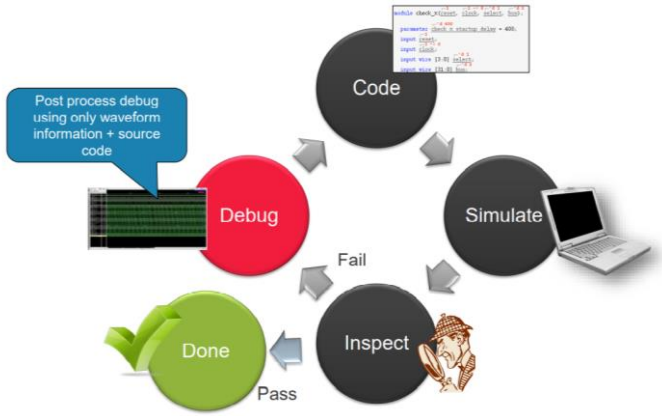


This slide shows the snapshots for interactive debug where you apply break points and try to pause simulation and debug and then continue as needed and repeat the iteration if required.

What Is Post Processing Debug?

This is the debug process done after you run the simulation. You open the waveform in your designated debug tool (SimVision) and then pull out all the simulation signals and objects and perform debugging.

- Post Processing Mode (inspection of HDL/HVL Waveforms)
- Waveform signal comparison (using simcompare)
- Driver tracing
- Probing assertions
- Probing HVL objects (transaction recording)
- Probing UVM hierarchy
- Sequence debug



11

cadence

This slide talks about what is post processing debug.

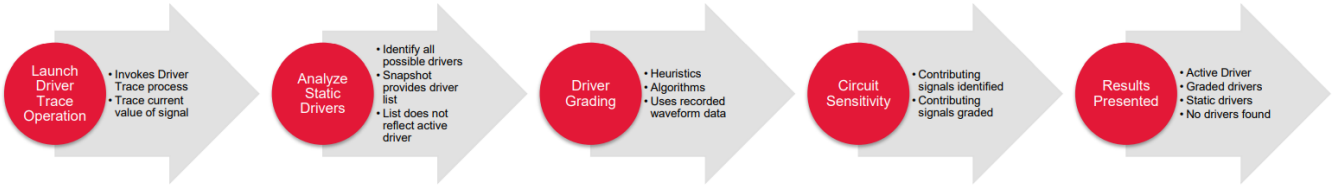
This is the debug process done after you run the simulation. You open the waveform in your designated debug tool like SimVision or Indago™, and then pull out all the simulation signals and objects and perform debug

In here we use:

- Post Processing Mode that is the inspection of HDL/HVL Waveforms)
- Waveform signal comparison (using simcompare)
- Driver tracing
- Probing assertions
- Probing HVL objects (transaction recording)
- Probing UVM hierarchy
- Sequence debug

Post Processing Debug Mode

- Waveform signal comparison using the (Sim)Compare Manager in the (SimVision) debug tool.
- Driver tracing
  - There are several steps that take place under the hood when users perform a driver trace operation within the debug tool:



- Probe assertions in several different ways
  - If assertions are present in an already probed module:
    - Assertion state as well as checked_count, finish_count, failure_count and disabled_count counters probed.
    - Contributing signals can be shown (if recorded and if the snapshot is loaded).

12

cadence

The different sub-processes or the steps that constitute a Post Processing Debug Mode are mentioned here.

Step 1 is a Waveform signal comparison using the (Sim)Compare Manager in the (SimVision) debug tool.

Step 2 is a Driver Tracing which consists of several substeps, which are:

- Invoke the driver.
- Identify all possible drivers. Use driver grading with heuristics and algorithms
- Identify the contributing signals and present the results

Step 3 is probing assertions in several different ways.

If assertions are present in an already probed module, then:

- Assertion state as well as checked-count, finish-count, failure-count, and disabled-count counters are probed.
- And contributing signals can be shown.

Post Processing Debug Mode (continued)

Probing HVL objects allow for debug at several levels of abstraction

- HDL signal level
- Transaction level
- Testbench level

Transaction recording is built in for UVM sequences

- No need to use the Cadence UVM library
- Must be enabled
- Easiest way to enable transaction recording for all objects

Probing UVM hierarchy

- Probing the entire UVM hierarchy
- Probing only a single UVC
- Users must request UVM base class fields via the –uvm switch

cadence

Continuing more info on Post Processing Debug Mode:

Probing HVL objects allow for debug at several levels of abstraction like at:

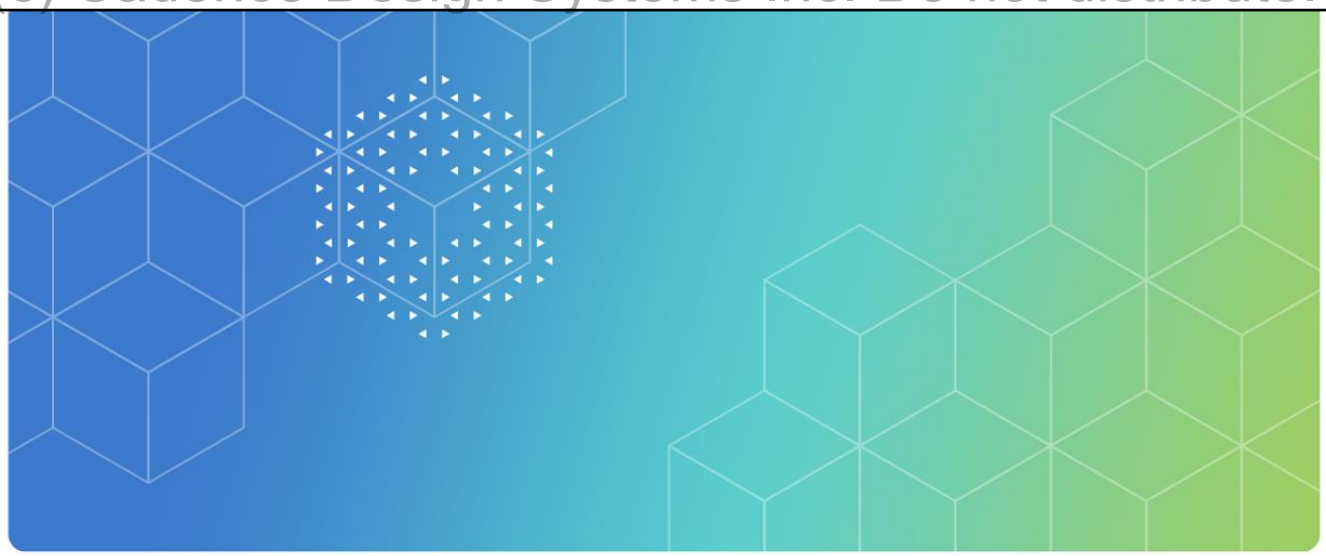
- HDL signal level
- Transaction level
- And at the testbench level

Transaction recording is built in for UVM sequences.

- No need to use the Cadence UVM library
- They must be enabled
- The easiest way to enable transaction recording for all objects

For probing UVM hierarchy, we can have different types like:

- Probing the entire UVM hierarchy
- Probing only a single UVC
- And users must request UVM base class fields via the –uvm switch



Module 9-3

Debugging Using Interactive Debug with Xcelium™: GUI and Textual/Batch Commands

cadence

Debugging with Xcelium using both GUI and textual or batch commands.

Objective

In this module, you will:

- Debug using the interactive debug technique.

In this submodule, you will debug using the interactive debug technique.

Scenario: Drink Machine Design

The drink machine example is a simple design and testbench used extensively throughout the Xcelium/SimVision documentation.

There is an error in the drink machine logic. It sometimes does not reset to the *idle* state when a drink is dispensed.

- All the source files for this design are included in the Cadence® installation hierarchy in the following directory:
  - *install_dir/doc/xsctutorial/examples*
- The design itself is made up of the following modules:
  - **Drink_machine**: Counts the amount of change that the user has entered, dispenses a drink, and returns any due change.
  - **Coin_counter**: Loads the machine with coins and determines when the device is out of change.
  - **Can_counter**: Loads the machine with drinks and determines when the machine is empty.
  - **State_machine**: Models the behavior for accepting coins and dispensing drinks:
    - Amount of money that the user has deposited so far defines the current state.
    - The type of coin the user deposits determines the machine's next state.

cadence

This page does not contain notes.

Scenario: Drink Machine State Table

A drink costs 50 cents.

The machine accepts:

- Nickels (5 cents)
- Dimes (10 cents)
- Quarters (25 cents)

The FSM must count coins until the cost is met or exceeded

- Then dispense a can...
- ...and return any change

The machine also keeps track of:

- Number of stored coins for change
- Number of cans

Current State	Transition Value	Next State (Change)
idle 4'd0	nickel_in dime_in quarter_in	five ten twenty_five
five 4'd1	nickel_in dime_in quarter_in	ten fifteen thirty
ten 4'd2	nickel_in dime_in quarter_in	fifteen twenty thirty_five
fifteen 4'd3	nickel_in dime_in quarter_in	twenty twenty_five forty
twenty 4'd4	nickel_in dime_in quarter_in	twenty_five thirty forty_five
twenty_five 4'd5	nickel_in dime_in quarter_in	thirty thirty_five idle
thirty 4'd6	nickel_in dime_in quarter_in	thirty_five forty idle (nickel_out)
thirty_five 4'd7	nickel_in dime_in quarter_in	forty forty_five idle (dime_out)
forty 4'd8	nickel_in dime_in quarter_in	forty_five idle idle (nickel_out, dime_out)
forty_five 4'd9	nickel_in dime_in quarter_in	idle idle (nickel_out) idle (two_dime_out)

17

cadence

The table shows the various states the drink machine can go through.

For example, when no money has been deposited, the machine is in an *idle* state. When the user adds a nickel, the machine transitions to the next state, *five*. When the current state is *five*, and the user adds a quarter, the machine transitions to the next state *thirty*. When the user adds 50 cents, the machine dispenses a drink and transitions back to the *idle* state. When the user adds more than 50 cents, the machine dispenses a drink, returns the correct change, and transitions back to the *idle* state.

Summary of the Problem

- **Problem**
  - There is an error in the drink machine logic. It sometimes does not reset to the **`IDLE`** state when a drink is dispensed.
- **Given**
  - Drink machine design and the testbench.
  - SimVision tool and Xcelium simulator.
- **Goal**
  - To use your preferred text editor to fix this error.
  - To rerun the simulation to ensure that the behavior is correct.

This page does not contain notes.

Possible Solutions

	Strategy	Pros	Cons
1.	Use Debug GUI	More visual, easy to view the waveform to figure out the error	Takes time to load
2.	Use Debug batch commands	Quicker and efficient time-wise	Visual is lacking

We shall primarily follow the SimVision GUI flow.

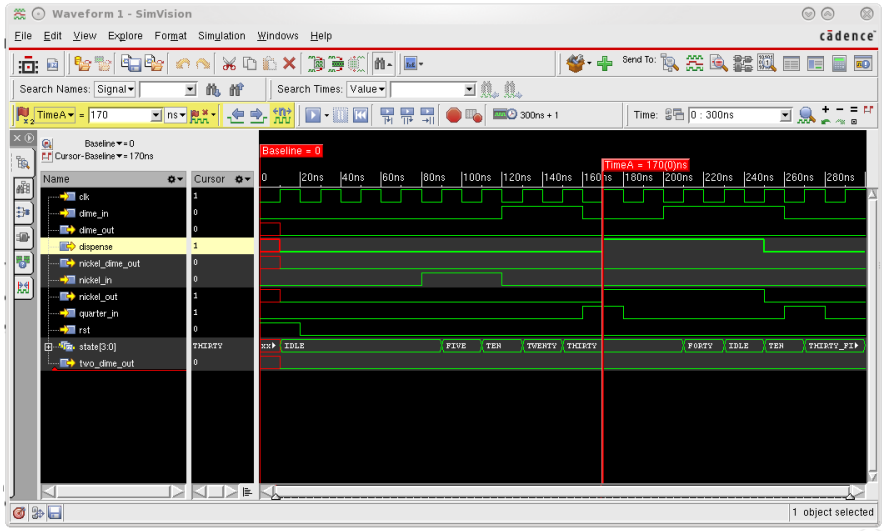
At the same time, show the textual batch commands that can be used to achieve the same.

Apart from the SimVision tool, the Cadence Indago™ Debug GUI tool could also be used for this Debugging Task.

This page does not contain notes.

How to Fix the Problem

- Use the waveform window to show how the values of signals change during simulation and locate where the error occurs.
- From the waveform window, go to the cause of the signal transition in the source browser.
- In the source browser, find this logic error.



At 170ns, **`dispense`** is 1, and the machine returns a nickel in change, but the **`state`** variable is not set to **`IDLE`**. Its value stays at **`THIRTY`** until the next coin is added, which is incorrect behavior.

cadence

This page does not contain notes.

Starting the Simulator Run Using the *xrun* Command

```
$ ls
dkm.sv  dkm_test.sv  run.f
$ xrun -f run.f
```

```
// run file for FSM
// files to be compiled
dkm.sv
dkm_test.sv

// command line options
-gui
-access rwc
-linedebug
-timescale 1ns/1ns
```

It is easiest to invoke Xcelium using a run file.

The command-line options are:

- **`-access rwc`** allows read, write, and connectivity access to the design objects. Required to probe signals and display their waveforms in SimVision.
- **`-gui`** invokes the SimVision Graphical User Interface.
- **`-linedebug`** enables support for setting line breakpoints and single-stepping through the code.
- **`-timescale 1ns/1ns`** sets the default time unit and precision information for the simulator.

cadence

This page does not contain notes.

The Terminal Displays the *xcelium* Prompt

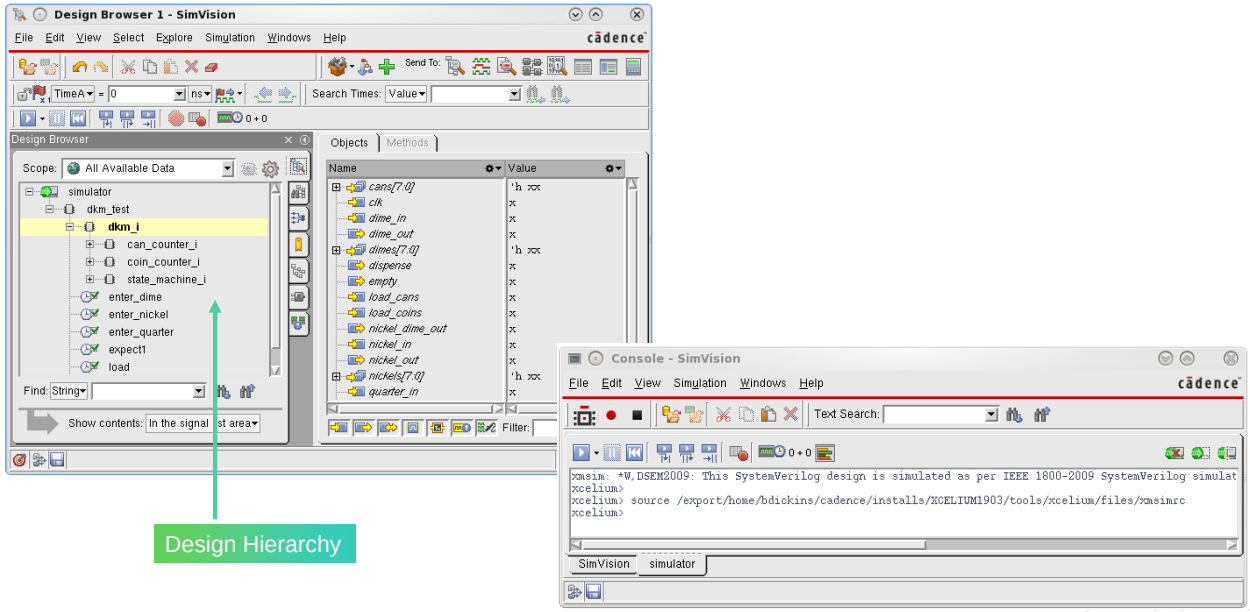
```
        module worklib.dkm:sv
                errors: 0, warnings: 0
file: dkm_test.sv
        module worklib.dkm_test:sv
                errors: 0, warnings: 0
                Caching library 'worklib' ....... Done
        Elaborating the design hierarchy:
        Top level design units:
                dkm_test
xmelab: *W,DSEMEL: This SystemVerilog design will be simulated as per IEEE 1800-2009 SystemVerilog simulation semantics. Use
tics.
        Building instance overlay tables: ..................... Done
        Generating native compiled code:
                worklib.state_machine:sv <0x2df9ba12>
                        streams:  55, words: 15263
                worklib.can_counter:sv <0x2fc2d616>
                        streams:   5, words:  1704
                worklib.coin_counter:sv <0x6778750d>
                        streams:  20, words:  7580
                worklib.dkm:sv <0x6ba8a9a6>
                        streams:  12, words:  3904
                worklib.dkm_test:sv <0x48b69bdb>
                        streams:  47, words: 24717
        Building instance specific data structures.
        Loading native compiled code:     ..................... Done
        Design hierarchy summary:
                                Instances  Unique
                Modules:                5       5
                Registers:             41      41
                Scalar wires:          42       -
                Vectored wires:         3       -
                Always blocks:          4       4
                Initial blocks:         1       1
                Cont. assignments:      2       2
                Pseudo assignments:    29      29
                Simulation timescale:  1ns
        Writing initial simulation snapshot: worklib.dkm_test:sv
xmsim: *W,DSEM2009: This SystemVerilog design is simulated as per IEEE 1800-2009 SystemVerilog simulation semantics. Use -dis
.

-----------------------------------
Relinquished control to SimVision...
xcelium>
xcelium> source /export/home/bdickins/cadence/installs/XCELIUM1903/tools/xcelium/files/xmsimrc
xcelium>
```

The terminal from which we invoke the *xrun* command also displays the *xcelium* prompt.

cadence

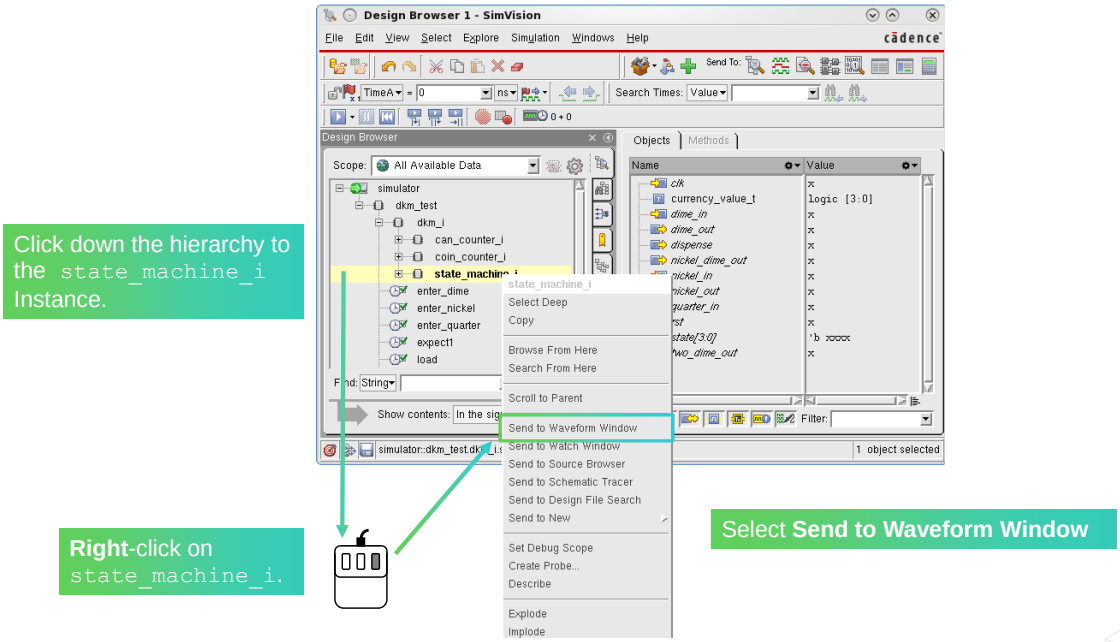
This page does not contain notes.

Opening the Design Browser and Console Window at Startup



When SimVision starts, the Design Browser and the Console windows appear. You access your design hierarchy in the Design Browser and enter the SimVision and simulator commands in the Console window.

Adding Variables to the Waveform Window



Click down the hierarchy to the `state-machine-i` instance.

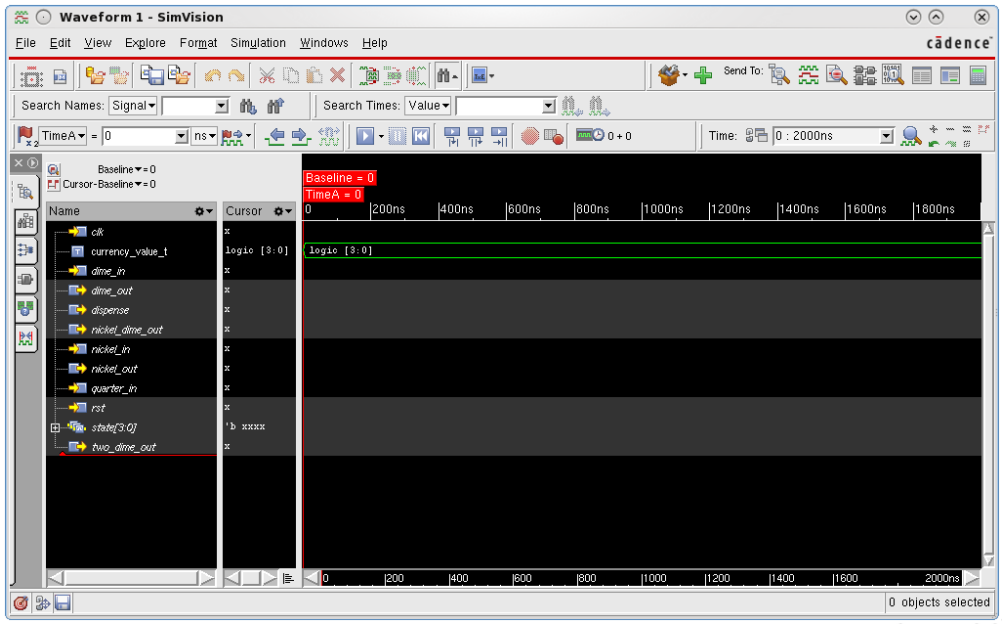
Right-click on the state-machine-I.

Select the **Send to Waveform** option.

Console Window Displaying Textual Commands

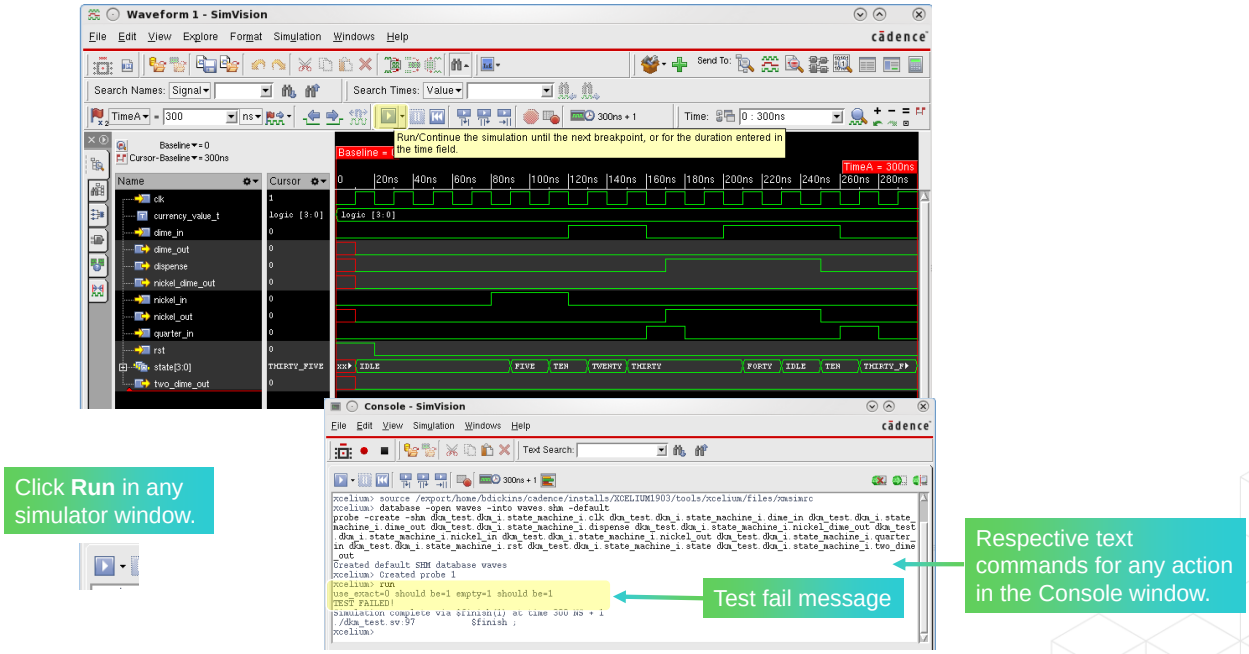
SimVision opens a Waveform window and adds all variables under `state_machine_i`.

Add variables to the waveform **before** running simulation.



SimVision displays the simulator messages in the Console window and the first command executed in the console window creates a simulation database and probes the signals that you added to the waveform window.

Running the Simulation

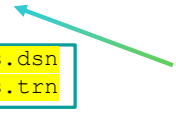


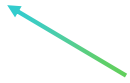
SimVision runs the test until it reaches a breakpoint or until it reaches the end of the test, and it saves the simulation data to the database.

The Waveform window is also updated during the simulation to display the signal transitions.

Waveform Database Creation

```
$ ls
dkm.sv  dkm_test.sv  run.f  waves.shm  xcelium.d  xrun.history  xrun.key  xrun.log
$ ls -hl waves.shm/
total 8.0K
-rw-r--r-- 1 ... 64K Oct  1 14:31 waves.dsn
-rw-r--r-- 1 ... 828 Oct  1 14:31 waves.trn
```



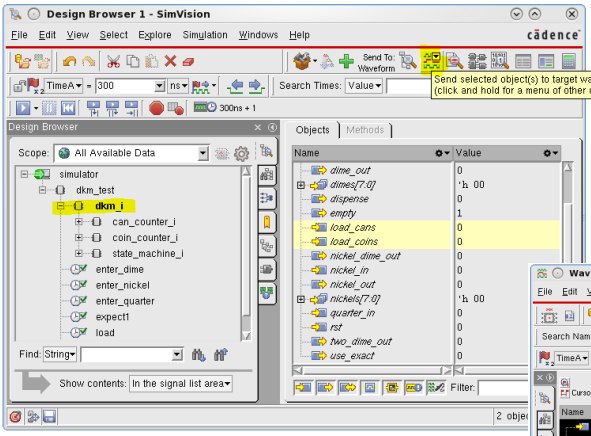


1. When the simulation run has been completed, your working directory should contain a new directory named `waves.shm`.
2. The `waves.shm` dir should contain 2 files: `waves.dsn` and `waves.trn`.
3. If these files are significantly smaller than 64k and 828 bytes, respectively, you may not have probed all of the required variables or run the simulation to the end.

cadence

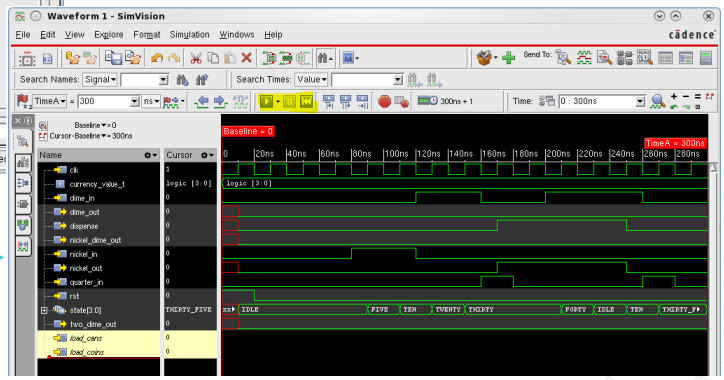
This page does not contain notes.

Adding More Relevant Signals to the Waveform Window



You can add more variables to the waveform by selecting and adding from the menu as before.

Alternatively, select the variable and then select the **Send to Waveform** icon.



Reset and rerun the simulation to see waveforms for the newly added variables.

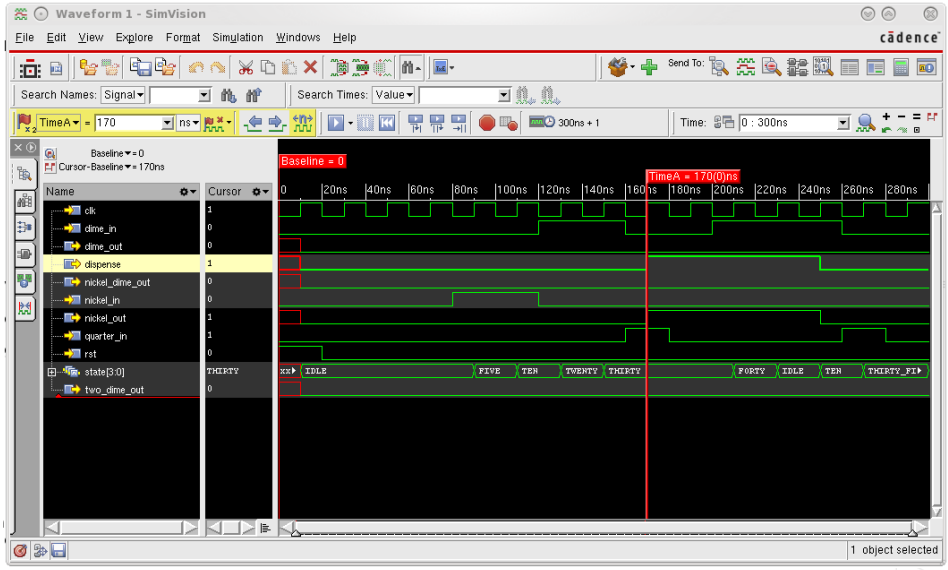
28 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Finding the Error in the Waveform Window

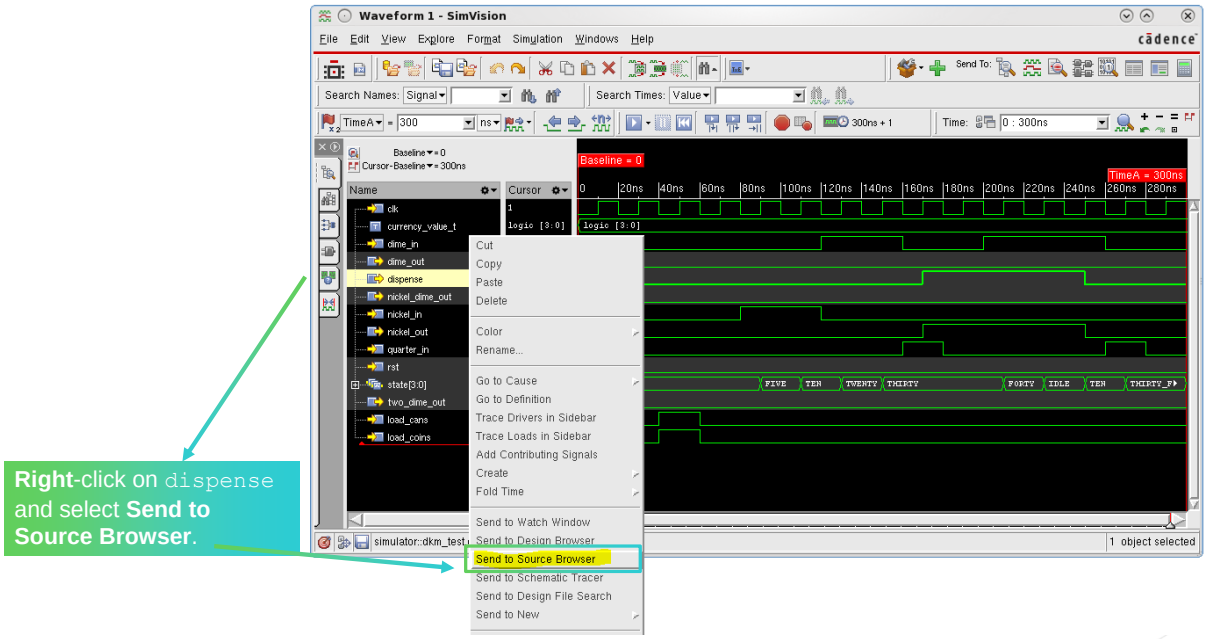
1. Move the Time cursor to the beginning of the simulation time by clicking at time **0** in the waveform.
2. Select `dispense`, then click **Next Edge** until the value of `dispense` is 1 at around 170ns.
3. Immediately prior to that time, `state` has the value `THIRTY`, and a quarter is added to the machine.
4. At the current cursor, `dispense` is 1, and the machine returns a nickel in change, but the state variable is not set to `IDLE`. Its value stays at `THIRTY` until the next coin is added, which is incorrect behavior.



cadence

This page does not contain notes.

Opening the Source Browser Window



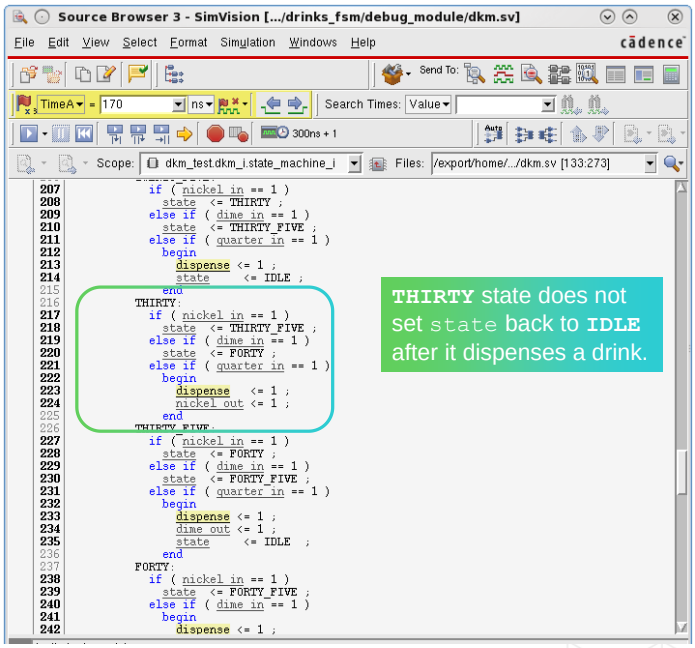
30 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Find the Cause of the Issue in the Source Browser

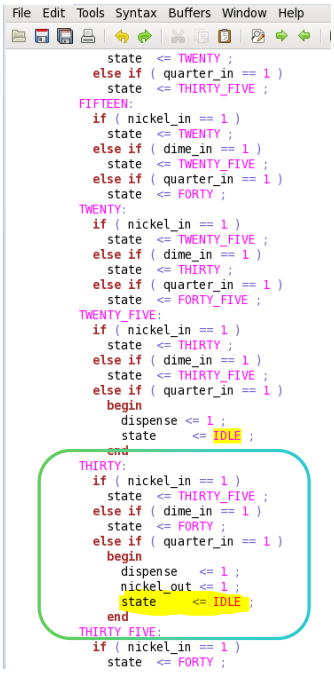
- Scroll down to the `THIRTY` state in the FSM.
- Once a drink has been dispensed, the FSM state should go to `IDLE`.
  - So, the FSM can start counting coins for the next delivery.
- The case statement for the `THIRTY` state does not set `state` back to `IDLE` after it dispenses a drink.



This page does not contain notes.

Fixing the Error

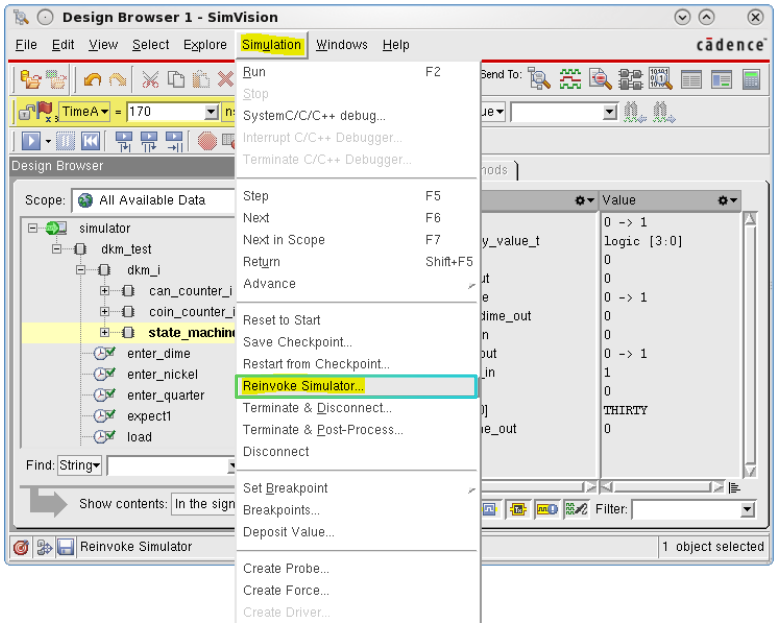
1. Open `dkm.sv` in your favorite editor.
2. Navigate to the `THIRTY` case branch in the `state_machine` module.
3. Set state to `IDLE` when dispense is set to 1.
4. Save the file!



cadence

This page does not contain notes.

Rerunning the Simulation from Any Window



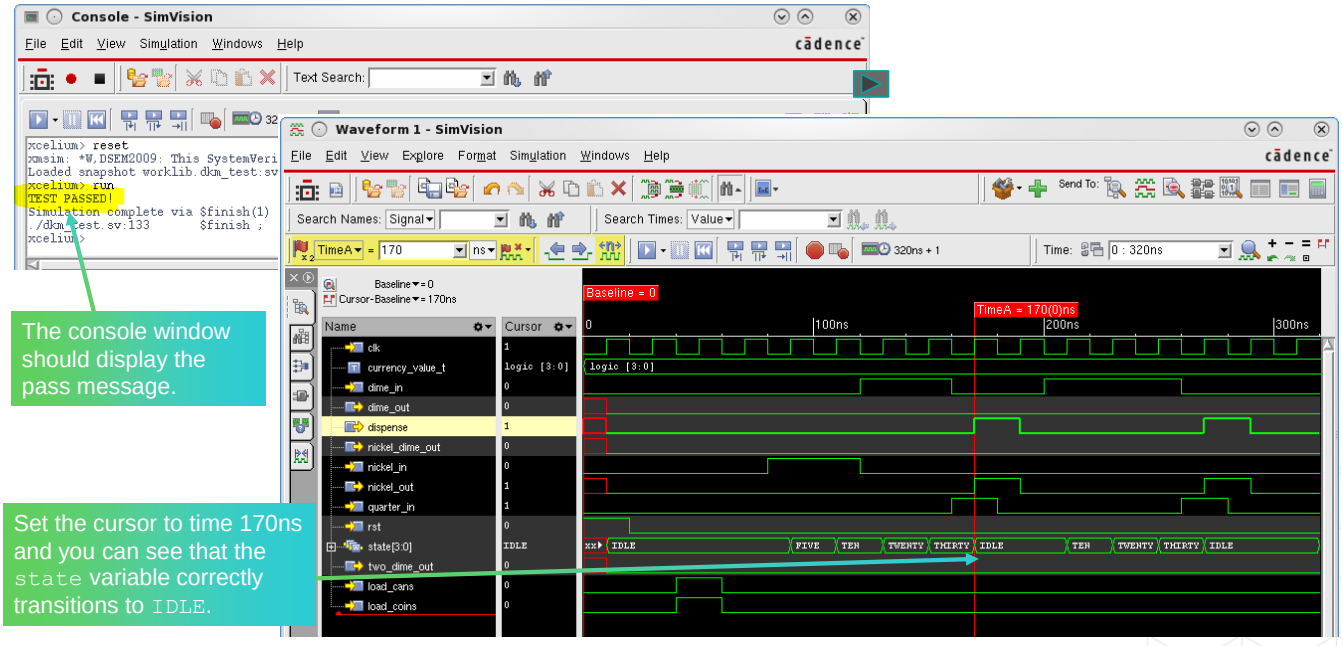
Select **Simulation –> Reinvoke Simulator** to recompile with your modified code.

If there is compilation error, fix the error and Reinvoke again.

cadence

This page does not contain notes.

Viewing the Passed Results



This page does not contain notes.

Summary of Problem and Solution

- **Error/problem**
  - There is an error in the drink machine logic. It sometimes does not reset to the `IDLE` state when a drink is dispensed.
- **Given**
  - Drink machine design and the testbench.
  - SimVision tool.
- **Goal**
  - To use your preferred text editor or the one that SimVision opens to fix this error.
  - To rerun the simulation to ensure that the behavior is correct.
- **Strategy**
  - Perform debug using the SimVision GUI, fix the code using your favorite editor, and rerun the design.
- **Results**
  - Error fixed and the drink machine design works as it should.

cadence

This page does not contain notes.